

SE3092 - Platform Based Development Assignment 01 | 2026 Semester 01 Te

Document FinPilot / FinanceTrack Personal Finance Management System for Android Submitted to
Sri Lanka Institute of Information Technology Faculty of Computing Prepared for the Kavindu
Silva personal finance scenario Technology Stack: Kotlin, Jetpack Compose, MVVM, Firebase,
Room Basic project-specific design document generated from the current implementation.

Table of Contents Introduction Scenario Analysis Problem Understanding Fu

Requirements Non-Functional Requirements and Constraints Technology Stack MVVM Architecture
Component Interaction Navigation Architecture Firestore Data Model Local Room Database and
Offline Strategy Security Rules and Data Ownership UI/UX Design Principles Annotated
Wireframes Design Justifications Implementation Status and Limitations Conclusion This
document is intentionally aligned with the SE3092 assignment brief. It avoids generic
budgeting-app language by grounding requirements in Kavindu Silva's irregular income,
low-friction expense entry needs, and MacBook Pro savings goal.

1. Introduction 1.1 Purpose This Technical Design Document describes the d

an Android personal finance management application developed for the SE3092 assignment. It converts the scenario of Kavindu Silva into a technical solution using Kotlin, Jetpack Compose, MVVM, Firebase Authentication, Cloud Firestore, Room, Retrofit, WorkManager, and Koin. 1.2 Application Overview Kavindu earns from a fluctuating salary, freelance projects, USD-based AdSense revenue, and crypto activity. He struggles to understand total monthly income, committed versus discretionary spending, and whether he is moving toward his LKR 490,000 MacBook Pro M4 goal. FinPilot addresses these problems by combining multi-source income recording, fast expense logging, dashboard analytics, offline-first transaction persistence, and goal tracking. 1.3 Scope Firebase Authentication with email/password, password reset, and Google Sign-In. Multi-currency income entry with live LKR conversion. Quick expense entry with committed and discretionary categorisation. Dashboard totals for income, expenses, net balance, spending overview, and recent transactions. MacBook-oriented savings goal tracking and manual savings boosts. Room-first local transaction persistence with background Firestore sync.

2. Scenario Analysis 2.1 Persona Summary Attribute Scenario Detail Design

Kavindu Silva, 25-year-old junior software engineer in Colombo. The app must feel lightweight, modern, and useful under real working pressure. Income Salary, freelance projects, AdSense USD, crypto gains/losses. Income records must store source, currency, exchange rate, and LKR equivalent. Expenses Rent, transport, food delivery, subscriptions, coffee, cash spending. Expense entry must be fast and broad enough to support month-end insight. Goal MacBook Pro M4, approximately LKR 490,000 within 12 months. The goal must be visible and connected to daily financial behaviour. Failed Attempts Sheets, generic budgeting apps, notes, bank summaries. The solution must reduce manual work and avoid rigid single-income assumptions. 2.2 Core Insight Kavindu's problem is not only missing records. It is financial invisibility. He lacks a simple place where irregular income, spontaneous expenses, and goal progress become visible together. The system therefore prioritises fast capture, normalised LKR calculations, and an at-a-glance dashboard over complex budget planning.

3. Problem Understanding 3.1 Income Structure The system must understand

fixed monthly salary only. It must support different sources, different currencies, and irregular arrival dates. Each income entry stores the raw amount, original currency, exchange rate used at entry time, and converted LKR amount. This makes totals historically stable and avoids recalculating past income with a new exchange rate. 3.2 Expense Friction Kavindu abandoned previous tools because they required too much effort. FinPilot's expense flow uses quick amount chips, category icons, date shortcuts, and optional notes. This reduces the number of decisions needed when logging a transaction in daily life. 3.3 Goal Visibility The MacBook goal should not be a separate spreadsheet. It is shown in the home/dashboard experience and can receive manual boosts or savings routed from income. This makes goal progress emotionally and financially visible. 3.4 Minimum Self-Awareness Views Net balance summary. Total income and total expenses. Spending categories, including committed and discretionary categories. Recent transactions. Goal progress and remaining amount.

4. Functional Requirements ID Requirement Priority Current Implementation

registration with email and password. High RegisterScreen, AuthViewModel, AuthRepository, FirebaseAuth. FR-02 User login with email and password. High LoginScreen and AuthRepository.login. FR-03 Password reset by email. Medium LoginScreen reset dialog and Firebase sendPasswordResetEmail. FR-04 Google Sign-In. Bonus LoginScreen Google flow and AuthRepository.signInWithGoogle. FR-05 Create and store user profile. High users collection, User model, ProfileScreen. FR-06 Record income from salary, freelance, crypto, or other sources. High AddIncomeScreen and IncomeViewModel. FR-07 Convert foreign income to LKR. High Retrofit ExchangeRateApi and FinanceRepository.getExchangeRate. FR-08 Record expenses quickly. High AddExpenseScreen and ExpenseViewModel. FR-09 Track MacBook savings goal. High GoalRepository, GoalDetailScreen, dashboard GoalCard. FR-10 Show dashboard summary and recent transactions. High HomeScreen, DashboardScreen, DashboardViewModel. FR-11 Show chronological transaction history. Medium TransactionHistoryScreen. FR-12 Support offline transaction capture. High Room database, FinanceRepository.saveTransaction, SyncWorker.

5. Non-Functional Requirements and Constraints ID Type Requirement / Con

Response NFR-01 Usability Daily logging must be low-friction. Quick chips, category icons, small forms, dashboard summaries. NFR-02 Reliability No blank screens or crashes on empty data. Loading, empty, and error states are used across major screens. NFR-03 Offline Transaction logging must not block on network. Room-first save, then background Firestore push. NFR-04 Security Users must only access their own data. Firebase Auth UID stored and checked in Firestore rules. NFR-05 Maintainability Clear separation of UI, ViewModel, Repository, and data models. MVVM with Koin, StateFlow, repositories, and models. NFR-06 Data Integrity Currency conversions must be historically stable. Each transaction stores exchangeRate and baseAmountLKR. CON-01 Platform Android API 26+ and target SDK 34+. minSdk 26 and targetSdk 35. CON-02 Language 100% Kotlin application layer. Kotlin source files and Gradle Kotlin DSL. CON-03 UI Jetpack Compose only; no XML layouts. All screens implemented as composables. CON-04 Backend Firebase Authentication and Firestore required. AuthRepository, DashboardRepository, GoalRepository, FinanceRepository.

6. Technology Stack Layer Technology Purpose Language Kotlin Primary ap

UI Jetpack Compose + Material 3 Declarative UI and modern Android components. Navigation Navigation Compose Single-activity screen navigation. Architecture MVVM + Repository Separation of UI state and data operations. State StateFlow Reactive UI state observed by composables. Async Kotlin Coroutines Non-blocking Firebase, Room, and API operations. Authentication Firebase Authentication Email/password, password reset, Google Sign-In. Cloud DB Cloud Firestore Remote storage for users, transactions, and goals. Local DB Room Offline transaction and recurring expense storage. Networking Retrofit + Gson Exchange-rate API integration. Background WorkManager Sync retries and recurring expense processing. Dependency Injection Koin Repository, ViewModel, Firebase, and Room wiring.

7. MVVM Architecture +-----+ | UI Layer - Jetpack Compose |

Register, Home, | | Income, Expense, Goal, | | Profile, History |

+-----+-----+ | v +-----+ | ViewModel Layer | |

AuthViewModel | | IncomeViewModel | | ExpenseViewModel | | DashboardViewModel | |

GoalDetailViewModel | | ProfileViewModel | +-----+-----+ | v

+-----+ | Repository Layer | | AuthRepository | | FinanceRepository

| | DashboardRepository | | GoalRepository | +-----+-----+ | v

+-----+ | Data Sources | | Firebase Auth | | Firestore | | Room

Database | | Retrofit Exchange API | +-----+ The UI layer only

displays state and forwards user events. ViewModels hold StateFlow objects and call

repositories using viewModelScope. Repositories contain business and data-access logic. The

data sources are Firebase Authentication, Firestore, Room, and Retrofit. Example: Login Flow

User enters email and password in LoginScreen. LoginScreen calls AuthViewModel.login(email,

password). AuthViewModel sets AuthState.Loading. AuthRepository calls

FirebaseAuth.signInWithEmailAndPassword. Repository loads the user profile from Firestore.

AuthViewModel emits AuthState.Success or AuthState.Error. LoginScreen observes the state and

navigates to Home or shows an error.

8. Component Interaction Feature UI ViewModel Repository/Data Source Auth

LoginScreen, RegisterScreen AuthViewModel AuthRepository, FirebaseAuth, users collection
Profile ProfileScreen ProfileViewModel AuthRepository, users collection Income
AddIncomeScreen IncomeViewModel FinanceRepository, GoalRepository, Retrofit, Room, Firestore
Expense AddExpenseScreen ExpenseViewModel FinanceRepository, Room, Firestore Dashboard
HomeScreen, DashboardScreen DashboardViewModel FinanceRepository, DashboardRepository,
GoalRepository Goal GoalDetailScreen GoalDetailViewModel GoalRepository, FinanceRepository
History TransactionHistoryScreen TransactionHistoryViewModel FinanceRepository and Room
transactions Background Sync Not visible Not applicable SyncWorker and FinanceRepository
Koin creates shared instances of FirebaseAuth, FirebaseFirestore, Room DAOs, repositories,
and ViewModels. This reduces manual wiring and keeps the codebase closer to the assignment's
MVVM requirement.

9. Navigation Architecture The application uses a single MainActivity and Na

Compose. A splash screen checks FirebaseAuth.currentUser and routes the user either to Login or Home. The home screen becomes the main launch point for dashboard, income, expenses, goals, profile, and transaction history. Splash |-- if not authenticated --> Login | |-- Register | |-- Forgot Password Dialog | |-- Google Sign-In | |-- if authenticated -----> Home |-- Dashboard |-- Add Income |-- Add Expense |-- Goal Detail |-- Transaction History |-- Profile / Settings Screen Routes splash login register home profile dashboard add_income add_expense transaction_history goal_detail/{goalId}

10. Firestore Data Model 10.1 users Collection Field Type Description id Strin

Auth UID. name String User display name. email String Authenticated email address.
baseCurrency String Default reporting currency, currently LKR. totalBalanceLKR Number Stored
balance field for profile/dashboard use. savingsPercentage Number Preferred percentage to
route toward savings. notificationsEnabled Boolean User setting for reminders. 10.2
transactions Collection Field Type Description userId String Owner UID. type String INCOME
or EXPENSE. amount Number Original user-entered amount. currency String LKR, USD, USDT, etc.
exchangeRate Number LKR conversion rate used at entry time. baseAmountLKR Number Normalised
amount used for totals. category String SALARY, FREELANCE, CRYPTO, COMMITTED, DISCRETIONARY,
SAVINGS. subCategory String Food, Transport, Coffee, Savings, etc. note String Optional
detail. timestamp Number Time of transaction.

11. Goals, Indexes, and Query Strategy 11.1 goals Collection Field Type Desc

String Owner UID. title String Goal name, e.g., MacBook Pro M4. targetAmount Number Goal target amount in LKR. currentSaved Number Current saved amount. deadline Number Deadline timestamp. status String ACTIVE or COMPLETED. 11.2 Indexing Strategy Collection Query Index Need transactions where userId == current user, orderBy timestamp desc Composite index may be required by Firestore. goals where userId == current user and status == ACTIVE Composite index may be required for efficient active-goal lookup. users document by UID No composite index needed. The implementation stores transactions in a flat top-level collection with a userId field. This supports chronological feeds and background sync, but requires security rules and userId filters to enforce ownership.

12. Local Room Database and Offline Strategy Room is used as the local per

The current Room database contains transaction and recurring expense tables. The key offline design is Room-first writes: when income or expense is saved, the record is inserted locally first and Firestore sync happens asynchronously.

12.1 Room Entities

Entity	Purpose	Important Fields
Transaction	Stores income and expense records locally.	id, userId, type, amount, currency, exchangeRate, baseAmountLKR, category, timestamp, isSynced.
RecurringExpense	Stores recurring expense templates.	id, userId, amount, category, interval, nextExecutionDate, isEnabled.

12.2 Sync Flow

User saves income or expense. FinanceRepository inserts the transaction into Room. A background coroutine attempts to upload the transaction to Firestore. If Firestore succeeds, the local row is marked `isSynced = true`. If network fails, the row remains unsynced. SyncWorker retries unsynced transactions when connectivity is available. This solves Kavindu's need to log expenses quickly even when mobile signal is weak, such as during travel.

13. Security Rules and Data Ownership Firestore rules are submitted in firestore

current rule structure allows authenticated users to access only their own profile document, and only transactions/goals whose userId matches their Firebase Auth UID. users/{userId} allow read/write if request.auth.uid == userId transactions/{transactionId} allow create if request.auth.uid == request.resource.data.userId allow read/update/delete if request.auth.uid == resource.data.userId goals/{goalId} allow create if request.auth.uid == request.resource.data.userId allow read/update/delete if request.auth.uid == resource.data.userId

13.1 Security Rationale Client-side checks are not enough because a malicious client can call Firestore directly. Server-side rules provide the final enforcement layer. Since financial records are private, every record must be bound to the authenticated user's UID. 13.2 Future Improvement Rules can be strengthened by validating required fields, allowed enum values, numeric ranges, and immutable owner fields. This would further reduce accidental or malicious invalid writes.

14. UI/UX Design Principles Principle Reason Implementation Low friction Ka

tools with heavy manual entry. Quick expense chips, icon categories, date shortcuts, simple forms. Financial clarity He cannot explain where money goes. Dashboard balance, spending overview, recent transactions. Goal visibility The MacBook goal was emotionally present but financially invisible. Goal card, progress bar, remaining amount, monthly need. Multi-currency awareness Income arrives in LKR, USD, and crypto-linked values. Currency selector and LKR conversion preview. Resilience Network failures should not stop logging. Room-first persistence and background sync. Modern Android Assignment requires Compose and Material 3. Material3 components, cards, buttons, dialogs, navigation. The design focuses on daily usability rather than advanced accounting. The first screen after login shows usable financial state and quick actions, so users do not need to hunt for the core workflows.

15. Annotated Wireframe 1 - Home / Dashboard Top Bar: user avatar and gre

Profile & Settings. Balance Card: net balance, total income, and total expenses in LKR. Goal Card: active MacBook goal, progress percentage, and boost actions. Quick Actions: Add Expense, Add Income, Transaction History, Goal Tracker. Spending Overview: category totals and visual progress indicators. Annotation Purpose A1 Balance card Immediately answers "how much money do I have after income and expenses?" A2 Goal card Keeps the MacBook Pro target visible every time the app opens. A3 Quick actions Reduces friction by making the main workflows one tap away. A4 Spending overview Gives a first-week understanding of where spending is concentrated.

16. Annotated Wireframe 2 - Add Income Amount Field: enter raw amount. C

LKR, USD, USDT. Income Type Selector: Salary, Freelance, Crypto. Conversion Preview: original amount approximately equals converted LKR amount. Save Income Button: saves transaction Room-first and syncs to Firestore. Smart Savings Dialog: optional routing of percentage toward active goal. This screen targets Kavindu's irregular and multi-currency income problem. Instead of requiring him to calculate LKR totals manually from multiple apps, the screen records the original amount and the converted baseAmountLKR in one flow.

17. Annotated Wireframe 3 - Add Expense Amount Field with LKR label. Quick

common amounts based on selected category. Category Grid: Food, Transport, Coffee, Bills, Health, Shopping, etc. Date Selector: Today, Yesterday, or Other. Recurring Toggle: auto-log future weekly/monthly payments. Optional Note Field. Add Expense Button. The expense screen is designed to be fast enough for repeated daily use. Broad committed and discretionary categories support the dashboard without forcing complex accounting behaviour.

18. Annotated Wireframe 4 - Goal Tracker and Profile Goal Header: goal title a

Progress Bar: percentage of target reached. Stats Cards: saved amount, target amount, months left, monthly need. Boost Buttons: LKR 500, 1000, 5000, or custom amount. Profile Settings: display name, email, base currency, savings percentage, reminders. Logout Button. The goal tracker makes the MacBook fund concrete by showing not only progress but also the monthly amount needed. The profile screen supports identity and financial preferences, including base currency and auto-savings percentage.

19. Design Justifications 19.1 MVVM with Repository MVVM suits Compose k

from state. ViewModels expose StateFlow, repositories hide Firebase/Room/API details, and composables remain focused on rendering and events. 19.2 Flat Transactions Collection Income and expense records are stored in a shared transactions collection. This supports one chronological feed and simple dashboard totals. The type field distinguishes income from expense. 19.3 Store baseAmountLKR Storing baseAmountLKR avoids recalculating historical values and keeps dashboard aggregation fast. The exchangeRate field preserves the conversion context used at the time of entry. 19.4 Room-first Persistence Room-first writes make the app responsive and offline-tolerant. Firestore sync happens later without blocking the user's immediate logging action. 19.5 Koin Dependency Injection Koin provides shared dependencies such as FirebaseAuth, FirebaseFirestore, Room DAOs, repositories, and ViewModels, reducing manual object creation and improving testability.

20. Implementation Status and Limitations 20.1 Implemented Firebase email/

authentication. Forgot password and Google Sign-In. Firestore-backed user profiles, transactions, and goals. Room local transactions and recurring expense templates. Income flow with live currency conversion. Expense flow with quick categories, dates, recurring setup, and notes. Dashboard and transaction history. Goal tracker and profile/settings. WorkManager sync and recurring expense workers. 20.2 Known Limitations Exchange-rate responses are not cached locally yet. Firestore rules currently check ownership but do not deeply validate field types. Some ViewModel wiring can be cleaned further for full DI consistency. Unit and instrumented test coverage is minimal. These limitations do not remove the core value of the application, but they are useful to mention honestly during a technical discussion.

21. Conclusion FinPilot is designed as a scenario-specific personal finance a

Silva. It focuses on the problems that caused previous tools to fail: multi-source income, invisible discretionary spending, low tolerance for manual entry, and a savings goal that was not connected to everyday behaviour. The implementation uses a modern Android stack: Kotlin, Jetpack Compose, Navigation Compose, MVVM, StateFlow, Coroutines, Firebase Authentication, Firestore, Room, Retrofit, WorkManager, and Koin. The architecture separates UI, ViewModels, repositories, and data sources while supporting offline-first transaction capture and background cloud sync. The system therefore meets the main technical constraints of the SE3092 assignment and provides a credible product that could improve Kavindu's financial clarity within the first week of use. End of Technical Design Document